LLM-generated SQL evaluation

For this analysis, I selected Gemini, Claude and ChatGPT .

Outputs

Gemini

To find the observer responsible, we need to join the `Personnel` table with the `Bird_nests` table, filtering for the specific site, date range, and the "float" aging method. By grouping by the observer's name and counting the nests, we can identify who hit that specific count of 36.

Here is the SQL query to solve the mystery:

```sql
SELECT
    p.Name,
    COUNT(bn.Nest_ID) AS Num_floated_nests
FROM Bird_nests bn
JOIN Personnel p ON bn.Observer = p.Abbreviation
JOIN Site s ON bn.Site = s.Code
WHERE
    s.Site_name = 'nome'
    AND bn.Year BETWEEN 1998 AND 2008
    AND bn.ageMethod = 'float'
GROUP BY p.Name
```

```
HAVING COUNT(bn.Nest_ID) = 36;
```

Breakdown of the Logic:

* **Joins:** We connect `Bird_nests` to `Personnel` to get the full names and to `Site` to filter by the human-readable site name ('nome').
* **Filters:** * `s.Site_name = 'nome'`: Restricts the search to the specific location.
* `bn.Year BETWEEN 1998 AND 2008`: Limits the time frame as specified by your colleague.
* `bn.ageMethod = 'float'`: Since the error involved floating eggs in salt water, we only look at nests where the "float" method was documented.

* **Aggregation:** We group the results by the observer's `Name` and use `HAVING` to filter for the exact count of 36 nests.

Claude

Looking at the schema, I need to find observers at the "nome" site between 1998–2008 who used the float method (`ageMethod = 'float'`), then find which one observed exactly 36 nests.

Here's the SQL:

```
SELECT
    p.Name,
    COUNT(*) AS Num_floated_nests
FROM Bird_nests bn
JOIN Personnel p
    ON bn.Observer = p.Abbreviation
WHERE
    bn.Site = 'nome'
    AND bn.Year BETWEEN 1998 AND 2008
    AND bn.ageMethod = 'float'
GROUP BY p.Name
```

HAVING COUNT(*) = 36;

How it works:

1. **`FROM Bird_nests`** — the nest records are the unit of analysis (one row per nest).
2. **`JOIN Personnel`** — to get the observer's full name from their abbreviation.
3. **`WHERE` filters** narrow to:
    - Site = `'nome'`
    - Years 1998–2008 inclusive
    - `ageMethod = 'float'` — only nests where egg age was determined by floating
4. **`GROUP BY p.Name`** — counts nests per observer.
5. **`HAVING COUNT(*) = 36`** — keeps only the observer with exactly 36 such nests, which returns exactly one row.

ChatGPT

Use `Bird_nests` for the nest records and `Personnel` to convert the observer abbreviation to the full name:

```sql
SELECT
    p.Name AS Name,
    COUNT(DISTINCT n.Nest_ID) AS Num_floated_nests
FROM Bird_nests AS n
JOIN Personnel AS p
    ON n.Observer = p.Abbreviation
WHERE n.Site = 'nome'
  AND n.Year BETWEEN 1998 AND 2008
  AND n.ageMethod = 'float'
GROUP BY p.Name
HAVING COUNT(DISTINCT n.Nest_ID) = 36;
```

This should return exactly one row: the observer’s full name and `36`.

LLM Evaluation

Gemini

Gemini's SQL is logically correct, but it shows weaker schema understanding because it uses COUNT(bn.Nest_ID) instead of COUNT(*). This suggests the model did not recognize that Nest_ID is a primary key and therefore never NULL. The joins and filters are correct, and the query will return the right observer, but the aggregation choice indicates a partial rather than deep understanding of the schema.

Claude

Claude's SQL uses COUNT(*), which demonstrates more understanding of the database schema that each row in Bird_nests represents exactly one nest and that Nest_ID is a non-nullable primary key. The query is minimal, clean, and fully aligned with the schema, which is showing a strong understanding of how the tables relate. Out of the three LLM outputs, this is the best one.

ChatGPT

ChatGPT's SQL is correct but it might be over-engineering the query rather than using schema-aware reasoning by using COUNT(DISTINCT n.Nest_ID). Since Nest_ID is already unique and the query does not join to tables that would duplicate rows, DISTINCT is unnecessary and indicates the model did not rely on schema knowledge. The logic and joins are correct, but the aggregation choice reflects a weaker understanding of the data schema.